Metadata-as-Data in Apache Hudi: A Multi-Modal Index Substrate for Lakehouse Tables

Prashant Wason
Apache Hudi PMC and Uber
San Francisco, USA
pwason@uber.com

Sagar Sumit
Apache Hudi PMC and Anyscale
Bengaluru, India
codope@apache.org

Sivabalan Narayanan
Apache Hudi PMC and Onehouse
San Francisco, USA
sivabalan@onehouse.ai

Abstract

As lakehouse tables scale toward millions of files and partitions, metadata itself becomes big data. Traditional big data systems try to reduce metadata to keep management tractable, at the cost of query performance; cloud data warehouses like Snowflake, BigQuery, and Redshift treat metadata as a managed, indexed structure, but within proprietary architectures. Apache Hudi brings this principle to the open lakehouse by treating *metadata as data*: an internal Merge-on-Read table whose partitions implement a multi-modal indexing subsystem (column statistics, bloom filters, record-level, secondary, and expression indexes), transactionally coupled with data commits and stored in an SSTable-like layout (HFile) tuned for the point and prefix lookups that dominate query planning. Concurrent indexing via a three-phase protocol backfills large indexes online while writers continue to commit. We describe the design and report results on both synthetic benchmarks and Uber production workloads: over 99% reduction in data scanned (secondary index on a table from TPC-DS SF10000 dataset), record-level index lookups 64× faster than global simple index at 36 billion-row scale with sub-millisecond per-key latency even at 400 billion+ rows, and 72% lower write latency on upsert workloads.

CCS Concepts

• **Information systems** → **Data management systems**; **Database design and models**.

Keywords

Table formats, metadata management, adaptive indexing, point and range lookups, cloud object storage, Apache Hudi

ACM Reference Format:
Prashant Wason, Sagar Sumit, and Sivabalan Narayanan. 2026. Metadata-as-Data in Apache Hudi: A Multi-Modal Index Substrate for Lakehouse Tables. In *Companion of the 2026 International Conference on Management of Data (SIGMOD/PODS '26), May 31–June 5, 2026, Bengaluru, India.* ACM, New York, NY, USA, 4 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 Introduction

Open table formats have become the transactional layer for lakehouse deployments on cloud object storage [7]. At large scale, metadata becomes the bottleneck: directory listings and bulk footer/stat reads dominate planning latency and API cost [6]. The scale of production deployments is striking: Uber manages 19,500+ datasets (350 PB, 6 trillion rows/day) on Apache Hudi [16], and ByteDance operates an exabyte-scale lake [9]. As systems reach this scale, metadata requires database-style techniques for storage, indexing, and maintenance [11].

In-file statistics such as Parquet column-chunk min/max values are necessary but insufficient: the total footer metadata is $O(\#files \times \#columns)$, and every footer must be opened to perform file selection [18]. For wide tables with thousands of columns, this forces planners to parse far more metadata than any single query requires. External index structures that decouple file selection from footer reads become necessary [13].

Apache Hudi addresses this by treating metadata as first-class data. Cloud data warehouses have long maintained metadata as a managed, indexed structure: Snowflake uses zone maps in a dedicated service [10], BigQuery stores distributed statistics tables [11], and Redshift keeps in-memory zone maps [8]. However, all three rely on proprietary, always-on metadata services coupled to a single engine. An open lakehouse has no such service: metadata must reside alongside data on object storage, remain readable by any engine, and stay consistent without a centralized coordinator. Hudi brings this principle to the open lakehouse: each table maintains an internal *metadata table*, co-located with data files, that stores file listings and multiple index modalities—replacing recursive listings and broad footer scans with index-based planning [3, 14]. Hudi further supports *concurrent indexing* to build large indexes online without pausing ingestion [2].

Contributions. We present a metadata-as-data architecture for open table formats: an internal MoR table whose partitions form a multi-modal index substrate with an HFile-based layout optimized for point and prefix lookups, and a concurrent indexing protocol that builds indexes online without blocking writers. We evaluate on TPC-DS and Uber production tables at up to 400 billion+ rows.

2 The Metadata Table

Organization. The metadata table is a Merge-on-Read (MoR) Hudi table where each metadata or index type is a distinct partition (see Table 1) [1, 3]. MoR is the natural fit because every data commit generates metadata updates; rather than rewriting base files on each commit, MoR appends delta logs and compacts in the background, ensuring that ingestion latency is not penalized by metadata maintenance [3]. Because transactional coupling, MoR compaction, and the HFile layout are shared infrastructure, adding a new index modality requires only defining a new partition—no changes to the write protocol or storage layer.

Transactional consistency. Every Hudi table maintains a *timeline*: a log of all actions (writes, compactions, clustering) at monotonically increasing instants [3]. The metadata table has its own timeline in a parent-child relationship with the data table's timeline: each data commit triggers a corresponding metadata commit at the same instant, keeping the two always in sync (Figure 1). A write is visible only when *both* the data and metadata commits complete,

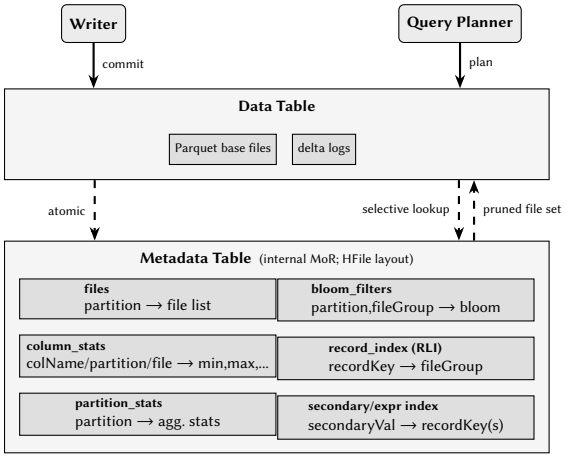

Figure 1: Metadata-as-data architecture and index substrate. Writers atomically commit data and metadata; query planners selectively consult metadata-table partitions for pruned file sets.

preventing partial exposure of data or index state under failures [14].

HFile for point and prefix lookups. Lakehouse metadata access is dominated by *point* and *prefix* lookups (e.g., "stats for column `c` in file `f`", "all blooms for partition `p`"). Hudi stores metadata partitions in an SSTable-like format (HFile), maintaining records in sorted order with a block index for point reads and range scans [4]. Published experiments report 10–100× lower point-lookup latency for HFile compared to Parquet/Avro at 10M-entry scale [14]. At Uber, the metadata table is deployed across 90% of production datasets; for the largest tables exceeding 300 billion rows, the record index alone is sharded into up to 10,000 HFiles [16].

Prefix key design for wide tables. For column statistics, Hudi composes record keys by concatenating *column name*, *partition name*, and *data file name*, enabling prefix scans. Planners issue prefix lookups only for columns referenced by a query, reducing work from $O(\#\textit{table columns})$ to $O(\#\textit{query columns})$ [14].

3 Multi-Modal Indexing

Hudi's metadata table acts as an *index substrate*: index modalities are materialized as partitions and can be enabled or disabled independently as workload needs evolve [1, 3]. Table 1 summarizes key modalities and their database analogues.

Record-level index. The record-level index (RLI) maps record keys to data-table locations in the `record_index` partition [1, 4]. It serves the same role as a primary index in a database: upserts and deletes no longer require full file-group scans to locate candidate records. To accommodate large key spaces, the record index uses hash-based sharding and supports point lookups for keyed reads [1, 17].

Column statistics and data skipping. The `column_stats` partition stores per-file, per-column aggregates (min, max, null count, value count) in the metadata table. Query planners evaluate predicates against these statistics to skip files whose value ranges provably exclude matching records—the same principle as zone maps in cloud data warehouses [8, 10, 11]. Coarse-grained statistics pruning composes with bloom filters and secondary indexes to further narrow the candidate file set.

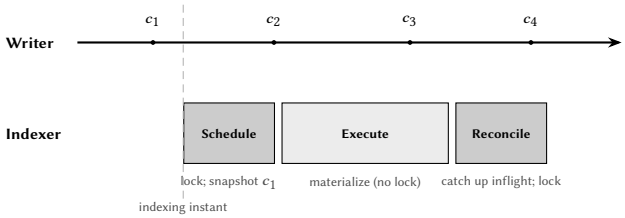

Figure 2: Concurrent indexing protocol. Schedule takes a brief lock and publishes an index plan; Execute materializes entries while writers proceed unblocked; Reconcile catches up inflight commits.

Expression index. An expression index indexes a function of a column, stored as an `expr_index` partition [1, 4]. The concept mirrors expression indexes in PostgreSQL [15], creating an access path for predicates that do not match raw columns—e.g., an hour-extraction index provides hour-level pruning on a day-partitioned table without changing the physical layout.

Adaptive bloom filters. Hudi supports *adaptive bloom filters* for membership testing during upserts, adjusting filter size per file to maintain a configured false-positive ratio [1]. This is useful for streaming and skewed workloads where file-group cardinalities vary and static filters would saturate.

Secondary index. The secondary index accelerates predicates on non-key columns through a two-hop lookup (secondary value → record key → file location), reusing RLI rather than maintaining a separate file-location mapping [4].

4 Concurrent Indexing

Indexes increase write cost because they must be maintained as data changes; in a continuously ingested lakehouse table, pausing writers to build a new index would violate ingestion SLAs. Traditional databases perform online index builds under row-level locks, but lakehouse writers operate over immutable file groups on object storage, where coordination must happen at the commit-metadata level. Hudi addresses this with asynchronous *concurrent indexing* that builds indexes while writers continue to commit (Figure 2) [2].

Concurrent indexing uses a three-phase protocol: (1) **Schedule** acquires a brief metadata-table lock, snapshots all completed commits up to an *indexing instant*, and publishes an index plan that subsequent writers will observe; (2) **Execute** materializes index entries from the snapshotted files without holding any lock—writers that commit after the indexing instant observe the published plan and include their own index updates in each commit, so they need no catch-up; and (3) **Reconcile** reacquires the metadata-table lock and catches up any writes that were inflight at the indexing instant—these are the only commits whose index entries were not captured by either the snapshot or the normal commit path.

Table 1: Index modalities as metadata-table partitions. Each modality maps to a familiar database concept and is implemented as a partition within the internal Hudi metadata table.

MDT partition / modality	DB analogue	Primary effect
`record_index`	primary index	fast upserts/deletes; point lookups
`secondary_index`	secondary (inverted) index	accelerate predicates on non-key columns
`expr_index`	functional index	accelerate predicates on expressions; aids partition evolution
`column_stats`	zone maps / min-max stats	file pruning via data skipping; wide-table selective reads
`partition_stats`	aggregated stats	coarse partition pruning without file scans
`bloom_filters`	membership filters	prune candidate files for existence checks / upserts
`files` (file listings)	catalog / manifest cache	avoid object-store listings; faster planning/writes

Table 2: Secondary index benchmark on TPC-DS (SF10000) `web_sales` (286K files, 7.2 billion rows).

Setting	Cold (s)	Warm (s)	Data read	Files read
No secondary index	252	204	759 GB	286,603
With secondary index	31	9	593 MB	150

Consistency invariant. Because each data commit records exactly which files were added or removed (§2), the indexer can derive index entries for the small number of inflight commits deterministically. If a conflict is detected (e.g., a commit rolled back between schedule and reconcile), the indexing attempt aborts and retries from a fresh snapshot. After the initial build, subsequent data commits maintain the index incrementally through the same transactional coupling, so the concurrent protocol is needed only for first-time index creation or full rebuilds.

5 Evaluation

Setup. The secondary index benchmark uses Hudi with Spark with 480 executors on the TPC-DS (SF10000) `web_sales` table (286K files, 7.2 billion rows) stored on S3; the query predicate is an equality filter on `ws_ship_customer_sk` with randomly chosen keys. The RLI lookup benchmark uses Spark with 1,000 executors on Uber production tables stored on GCS.

Secondary index. A secondary index on the `web_sales` table reduces data scanned by over 99% (759 GB to 593 MB) and files read from 286,603 to 150, improving warm-run query latency by 96% (Table 2).

Record-level index. Global simple index joins incoming keys against every file group, so its cost grows with table size. RLI resolves keys via point lookups into sharded HFiles (Table 3): at 36B rows the speedup is 64×, and at 400B+ rows RLI maintains sub-millisecond per-key latency while global simple fails to complete. On a separate 1 TB upsert workload, RLI also reduces write latency by 72% and data shuffled by 92% [17].

Production deployment. Of Uber's 19,500+ Hudi datasets (§1), 3,600 upsert-heavy tables rely on the record index in production, with per-key lookup latency of 1–2 ms from sharded HFiles [16]. Each record-index entry is fixed-size (50–70 bytes compressed). For one of the largest tables (549 TB data, 10K shards), the record index adds 40 TB (~7% of data size), and for small tables it is under 1%. The fixed per-record cost favors wide schemas: tables with many columns amortize index overhead over larger rows.

Table 3: Record-level index lookup on Uber production tables (100K-key batches). Global simple did not complete on the 400B+ table.

Records	Record Index	Global Simple	Speedup
1.5B	0.14 ms/key	0.90 ms/key	6×
36B	0.45 ms/key	29.0 ms/key	64×
400B+	0.89 ms/key	— (did not complete)	

6 Discussion

All three major open table formats address the metadata bottleneck differently; cross-format evaluations at enterprise scale exist [12], but indexing-specific benchmarks remain future work. Delta Lake consolidates file-level statistics in a transaction log with periodic Parquet checkpoints [6]. Iceberg organizes snapshots as a manifest hierarchy with per-file column statistics in manifests and Puffin sidecar files for bloom filters and sketches [5].

Hudi takes the metadata-management idea further: rather than a log or manifest tree, it stores metadata in an internal MoR table where each index modality is an independently managed partition [1, 3]. The HFile-based layout is tuned for point and prefix lookup patterns, and the concurrent indexing protocol (§4) allows building indexes online [2]. In enterprise data lakes where records flow through multiple transformation stages with upserts at each stage, such a substrate directly reduces the cost of locating and updating records across the pipeline. Multiple query engines (Spark, Flink, Presto, Trino) already access the metadata table through Hudi's connector layer, so the index benefits extend across the lakehouse engine ecosystem.

7 Conclusion

At exabyte scale, metadata management is a data-management problem. Hudi's metadata-as-data design stores metadata and indexes as a first-class internal MoR table, couples updates via multi-table transactions, organizes index modalities as independently maintained partitions, and uses an optimized layout for the point and prefix lookups that dominate query planning; concurrent indexing enables online backfill without pausing writers. Future directions include bitmap and vector indexes, cost-based index selection, and cross-format index interoperability. As tables grow, metadata must be treated as managed, indexed data; we argue that the metadata-as-data pattern provides a practical foundation for doing so.

References

[1] Apache Hudi Indexes 2024. *Indexes*. Retrieved March 7, 2026 from https://hudi.apache.org/docs/indexes

[2] Apache Hudi Indexing 2024. *Concurrent Indexing*. Retrieved March 7, 2026 from https://hudi.apache.org/docs/metadata_indexing

[3] Apache Hudi Metadata 2024. *Table Metadata*. Retrieved March 7, 2026 from https://hudi.apache.org/docs/metadata

[4] Apache Hudi Tech Spec 2024. *Apache Hudi Technical Specification 1.0*. Retrieved March 7, 2026 from https://hudi.apache.org/learn/tech-specs-1point0

[5] Apache Iceberg 2024. *Apache Iceberg Table Format Specification*. Retrieved March 7, 2026 from https://iceberg.apache.org/spec

[6] Michael Armbrust, Tathagata Das, Liwen Sun, Burak Yavuz, Shixiong Zhu, Mukul Murthy, Joseph Torres, Herman van Hovell, Adrian Ionescu, Alicja Łuszczak, Michał undefinedwitakowski, Michał Szafrański, Xiao Li, Takuya Ueshin, Mostafa Mokhtar, Peter Boncz, Ali Ghodsi, Sameer Paranjpye, Pieter Senster, Reynold Xin, and Matei Zaharia. 2020. Delta lake: high-performance ACID table storage over cloud object stores. *Proc. VLDB Endow.* 13, 12 (Aug. 2020), 3411–3424. doi:10.14778/3415478.3415560

[7] Michael Armbrust, Ali Ghodsi, Reynold Xin, and Matei Zaharia. 2021. Lakehouse: A New Generation of Open Platforms that Unify Data Warehousing and Advanced Analytics. In *11th Conference on Innovative Data Systems Research (CIDR 2021)*. www.cidrdb.org, Virtual Event, January 11-15, 2021, 8 pages. https://www.cidrdb.org/cidr2021/papers/cidr2021_paper17.pdf

[8] Nikos Armenatzoglou, Sanuj Basu, Naga Bhanoori, Mengchu Cai, Naresh Chainani, Kiran Chinta, Venkatraman Govindaraju, Todd J. Green, Monish Gupta, Sebastian Hillig, Eric Hotinger, Yan Leshinksy, Jintian Liang, Michael McCreedy, Fabian Nagel, Ippokratis Pandis, Panos Parchas, Rahul Pathak, Orestis Polychroniou, Foyzur Rahman, Gaurav Saxena, Gokul Soundararajan, Sriram Subramanian, and Doug Terry. 2022. Amazon Redshift Re-invented. In *Proceedings of the 2022 International Conference on Management of Data (SIGMOD '22)*. Association for Computing Machinery, New York, NY, USA, 2205–2217. doi:10.1145/3514221.3526045

[9] ByteDance. 2021. *Building EB-Level Data Lake Using Apache Hudi at ByteDance*. Retrieved March 7, 2026 from https://hudi.apache.org/blog/2021/09/01/building-eb-level-data-lake-using-hudi-at-bytedance

[10] Benoit Dageville, Thierry Cruanes, Marcin Zukowski, Vadim Antonov, Artin Avanes, Jon Bock, Jonathan Claybaugh, Daniel Engovatov, Martin Hentschel, Jiansheng Huang, Allison W. Lee, Ashish Motivala, Abdul Q. Munir, Steven Pelley, Peter Povinec, Greg Rahn, Spyridon Triantafyllis, and Philipp Unterbrunner. 2016. The Snowflake Elastic Data Warehouse. In *Proceedings of the 2016 International Conference on Management of Data (SIGMOD '16)*. Association for Computing Machinery, New York, NY, USA, 215–226. doi:10.1145/2882903.2903741

[11] Pavan Edara and Mosha Pasumansky. 2021. Big metadata: when metadata is big data. *Proc. VLDB Endow.* 14, 12 (July 2021), 3083–3095. doi:10.14778/3476311.3476385

[12] Samuel Guleff. 2023. *Lakehouse at Fortune 1 Scale*. Retrieved March 7, 2026 from https://medium.com/walmartglobaltech/lakehouse-at-fortune-1-scale-480bcb10391b

[13] Chunwei Liu, Anna Pavlenko, Matteo Interlandi, and Brandon Haynes. 2023. A Deep Dive into Common Open Formats for Analytical DBMSs. *Proc. VLDB Endow.* 16, 11 (July 2023), 3044–3056. doi:10.14778/3611479.3611507

[14] Sivabalan Narayanan and Ethan Guo. 2022. *Introducing Multi-Modal Index for the Lakehouse in Apache Hudi*. Retrieved March 7, 2026 from https://www.onehouse.ai/blog/introducing-multi-modal-index-for-the-lakehouse-in-apache-hudi

[15] PostgreSQL 2024. *Indexes on Expressions*. Retrieved March 7, 2026 from https://www.postgresql.org/docs/current/indexes-expressional.html

[16] Uber Engineering. 2026. *Apache Hudi at Uber: Engineering for Trillion-Record-Scale Data Lake Operations*. Retrieved March 7, 2026 from https://www.uber.com/blog/apache-hudi-at-uber

[17] Shiyan Xu and Sivabalan Narayanan. 2023. *Record Level Index: Hudi's blazing fast indexing for large-scale datasets*. Retrieved March 7, 2026 from https://hudi.apache.org/blog/2023/11/01/record-level-index

[18] Xinyu Zeng, Yulong Hui, Jiahong Shen, Andrew Pavlo, Wes McKinney, and Huanchen Zhang. 2023. An Empirical Evaluation of Columnar Storage Formats. *Proc. VLDB Endow.* 17, 2 (Oct. 2023), 148–161. doi:10.14778/3626292.3626298